# Automated Machine Learning (AutoML) Platform

Team Members:
M Charan (240150018)
Charan Ponaganti (240150024)
Lalit Deshmane (240150044)

Course Details: DA 213
Date: April 22, 2026

## 1 Overview

Building and deploying machine learning models often requires specialized programming knowledge, from data preprocessing to adjusting algorithm settings. This project presents a fully automated machine learning (AutoML) platform aimed at removing the burden of ongoing model tuning. The main goal is to let people with limited data science experience upload raw tabular datasets. The platform will automatically cleaan and process the features and provide a ranked cross-validation evaluation of various traditional machine learning models. It supports both regression and classification workflows. The platform focuses on offering an easy-to-use graphical interface for tracking feature distribution, conducting Exploratory Data Analysis (EDA). You can also use it to upload a empty file and predict the end results of tthose

## 2 System Architecture and Workflow

The application use a client-server architecture where frontend and backend works separately, and it updates automatically without needing to reload the page.

### 2.1 Frontend Architecture

The frontend of the application is built using React.js as a Single Page Application. It keeps track of data across different pages using browser storage where important information like model results and column details are saved. This helps in avoiding repeated API calls during EDA. The interface communicates with the backend by sending asynchronous requests using Axios..

### 2.2 Backend Architecture

The backend is built using FastAPI, which provide a simple REST API. It keeps the model data in memory while the app is running. Since training models can take a long time and may cause request timeouts, the training is done asynchronously. The app uses `ThreadPoolExecutor` to run these tasks in the background. It also uses `threading.Event` to check if the client disconnect and stops the process early to save resources.

### 2.3 Execution Workflow

The runtime journey is segmented into four distinct interactions:

1. **Inputs:** The user provides a CSV structure. The backend reads the header buffer, detects dimension constraints, and responds with parameter definitions.
2. **Target:** A target prediction column is identified.
3. **Training Phase:** Standardized data frames are handed off to Scikit-Learn pipelines. Background tasks encode and iterate models, computing scoring heuristics.
4. **Usage:** Users consume the resulting mathematical models by piping fresh data points through the active session predictor.

### 2.4 Additional Interface Features

**Feedback Form:** The platform includes a feedback form that allows users to submit their experience and suggestions after using the system which gets mailed to us so we can use those for future implementations.

**History Page:** A history page is implemented to store previously uploaded datasets, selected target variables, and model results. This enables users to revisit past experiments without training the data again ,so reducing time

## 3 Methodology and Approach Used

To construct generalized automated intelligence, the pipeline adapts raw data structures through rigid transformation rules.

### 3.1 Target Identification

The system decides whether the problem is classification or regression by looking at the target variable. If the target column is numeric but have less than 20 unique integer values, and the ratio of unique values to total rows is below 0.05, then the backend treats it as a classification problem.

### 3.2 Automated Preprocessing

Extra columns can affect the model performance. So the system removes ”redundant features”, mainly string columns that have more than 15 unique values like IDs or names. Missing numeric values are filled using median to avoid effect of outliers, while missing categorical values are replaced with a ”missing” label so the model can still learn from it. One-Hot encoding is used to convert categorical data into binary form which is suitable for training models.

### 3.3 Cross-Validation Strategy

Generalization guarantees are established via stratified cross-validation (up to 5 independent folds, scaling downwards dynamically based on the frequency of the minority target class). This approach calculates the Mean Absolute Error ($MAE$) and Coefficient of Determination ($R^2$) metric for sequential regression arrays, while classification evaluations return weighted $F1$ and base precision metrics.

## 4 Dataset Details

The application actively accepts any valid comma-separated variables (.csv) structure under a strict 50MB ceiling layer . The application also includes some preloaded datasets which you can use as follows

- **1000_Companies.csv:** A tabular financial ledger profiling startup funding versus geographic spending. It serves principally as a multiple linear regression anchor aiming to accurately map total profit boundaries.
- **titanic.csv:** The traditional passenger survival history utilized primarily to validate the classification boundaries generated under categorical imbalance.
- **ai_job_impact.csv:** An unstructured analysis representing automation domain shifts, heavily populated by categorical strings which validates the redundancy filtering systems.
- **iris.csv:** A builtin dataset about sepallength,width etc..

## 5 Implementation Details

The codebase integrates several open-source numerical libraries to compose the processing environments.

### 5.1 Algorithmic Models

The platform utilizes concurrent trial fits spanning decision tree ensembles and linear solvers.

- **Classification Space:** Evaluates discrete variants of Logistic Regression, Gradient Boosting Classifiers, robust Random Forests, AdaBoost arrays, Extra Trees, HistGradientBoosting, and non-parametric K-Nearest Neighbors.
- **Regression Space:** Operates continuous variants including Linear Models, Gradient Regressors, and Euclidean Nearest Neighbors matching.

### 5.2 Tools and Core Stack

- **Data Transformation:** `pandas` handles schema manipulation and `numpy` facilitates rapid vector quantization for EDA distributions.
- **Machine Learning Engine:** `scikit-learn` provides the underlying model topologies and native imputation transforms.

- **Visualization:** `recharts.js` interprets raw dictionary outputs (scatter matrices, feature importance scaling, Interquartile Range derivations) to paint interactive SVG plots safely on the client.

### 5.3 Additional Functional Components

- **Feedback Form:** Captures user queries and mails it to us for improving our system
- **History Page:** Maintains session-based or persistent logs of previous model runs.

## 6 How to Run the Project

The Project is already deployed using vercel which can be opened by `https://automl-one.vercel.app/` .
The deployment is divided into two parts, a local server and a local client. Make sure that Python 3.10+ and Node.js is installed properly on the system.

### 6.1 Backend Initialization

1. Open a terminal at the project root directory and navigate to the isolated backend.

```
cd backend
```

2. Install the strict environment dependencies.

```
pip install -r requirements.txt
```

3. Run the internal ASGI interface on port 8000.

```
uvicorn main:app --reload --port 8000
```

### 6.2 Frontend Initialization

1. Open a secondary terminal environment context and traverse to the React core.

```
cd frontend
```

2. Install the Node modules tree.

```
npm install
```

3. Spin up the webpack development server.

```
npm start
```

The unified platform automatically aggregates at `http://localhost:3000`.
This should make the application run.

# 7 Results and Outputs

Once the dataset goes through the system, the interface shows various scores across various metrics

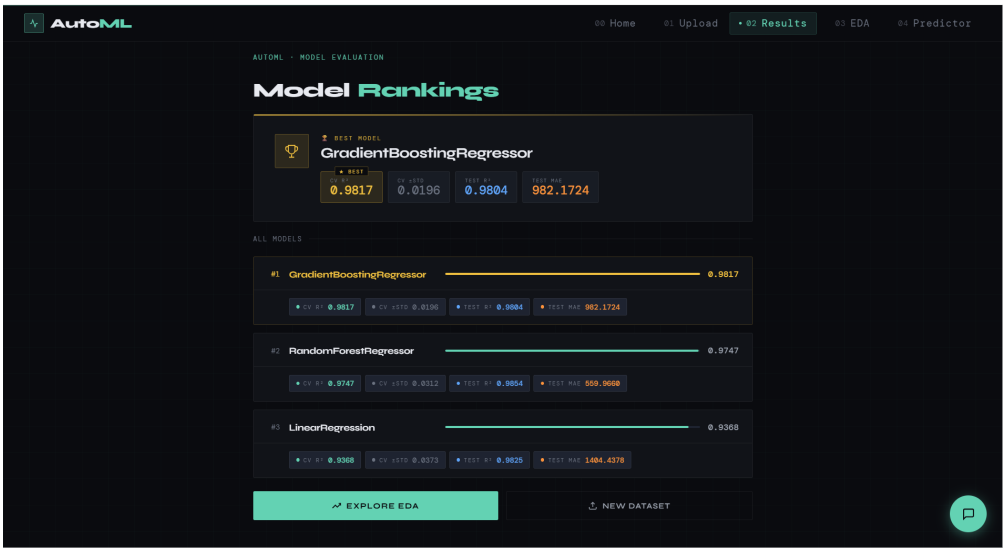


Figure 1: Model Rankings highlight the winning configuration based on CV mean metrics.

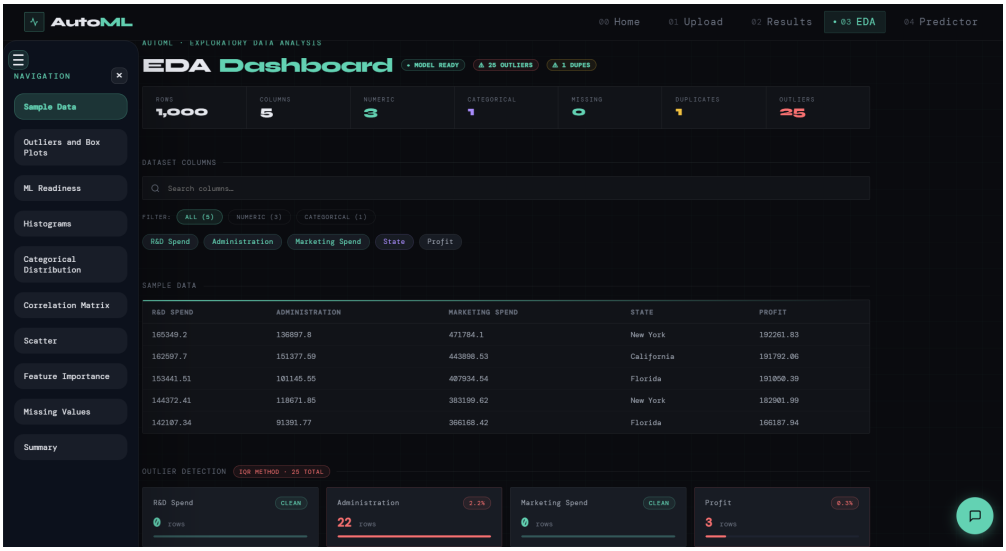


Figure 2: Exploratory Data Analysis tab providing deeper insights into feature distributions, correlations, and data characteristics to support model interpretation.

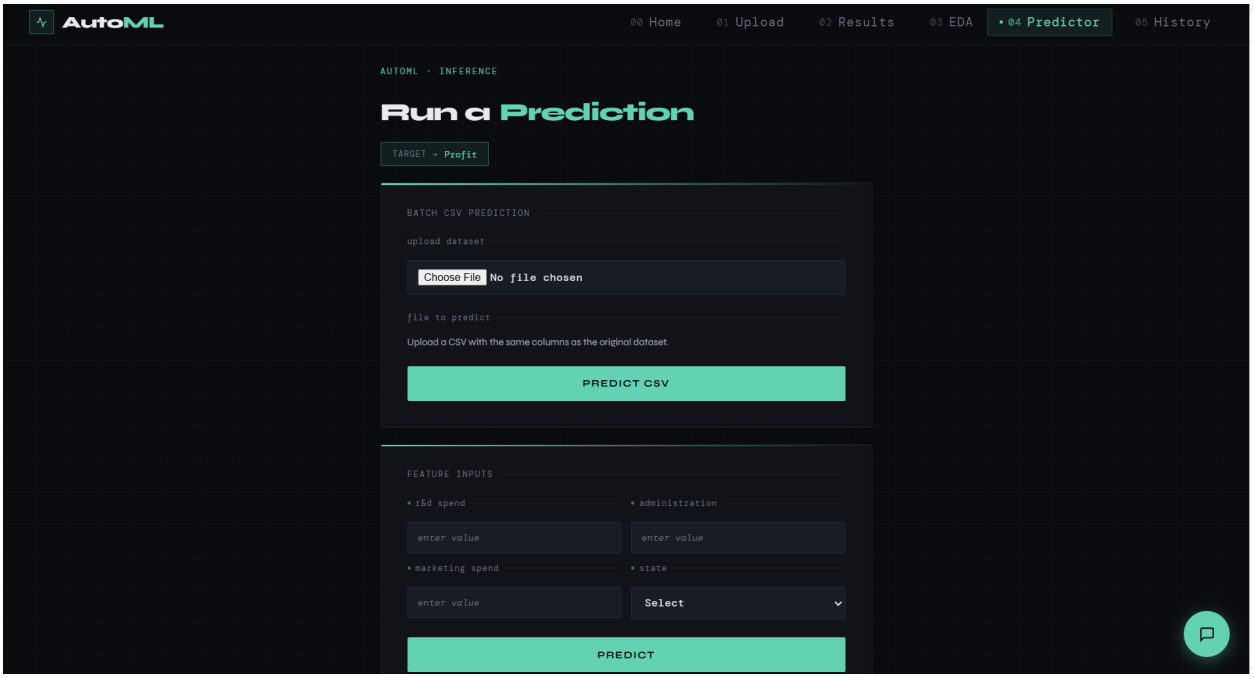


Figure 3: The **Predictor Module** surfaces interactive input schemas for single-row inference mappings, allowing endpoint outputs derived from the previously secured baseline algorithms.

## 8 Limitations and Future Scope

Right now, the platform has a few real limitations worth being upfront about. The biggest one is memory: keeping dataframes loaded directly in the web server works fine for small datasets, but once you're pushing

into the higher megabytes, it starts to crack.
The other issue is that the preprocessing pipeline is fairly rigid. It handles standard structured data reasonably well there's no NLP processing, no embeddings, nothing that handles sequential patterns. That's a real gap for anyone working with anything beyond clean tabular data. The plan is to move away from in-memory storage toward something like Redis or Postgres. That would let the platform handle concurrent workloads without everything bottlenecking through a single server process. It's a foundational change, but the right one. Hyperparameter tuning is also on the list. The current defaults get you somewhere, but smarter search methods
The last piece is portability. The goal is to let users export their trained pipelines as Docker containers rather than just .pkl files. That way, if someone wants to host their model outside the platform, they can — without needing to reverse-engineer the environment it was built in.

# 9 References


1. https://scikit-learn.org/0.21/documentation.html
2. https://fastapi.tiangolo.com/tutorial/
3. https://www.w3schools.com/REACT/default.asp
4. **For datasets** https://www.kaggle.com/